

LAB ASSIGNMENT REPORT

MODULE 2 – UNIT TEST FRAMEWORKS

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Practical / Lab Report

Software & Environment Requirements

Python 3, a code editor/IDE, PyTest, Selenium 4 and a supported browser are required for the Selenium-related experiments. Install the main packages using **`python -m pip install pytest selenium pytest-html`**. The unittest examples use Python's standard library and do not require an additional framework package.

Experiment 1 – Create and Execute a unittest Test Case

Aim: To create a basic Python unit test using the built-in unittest framework, perform assertions and execute the test runner.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a Python file named test_calculator_unittest.py.
2. Define a small Calculator class.
3. Create a TestCase class derived from unittest.TestCase.
4. Write test methods beginning with test_.
5. Use assertion methods to validate expected output.
6. Execute the file from the command line.

Source Code

```
import unittest

class Calculator:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b

class TestCalculator(unittest.TestCase):

    def setUp(self):
        self.calc = Calculator()

    def test_add(self):
        self.assertEqual(self.calc.add(10, 20), 30)

    def test_multiply(self):
        self.assertEqual(self.calc.multiply(5, 4), 20)

    def tearDown(self):
        self.calc = None

if __name__ == "__main__":
    unittest.main()

# Run:
# python test_calculator_unittest.py
```

Expected Result: Expected result: unittest discovers both test methods and reports them as passed.

Conclusion: This experiment demonstrates the basic TestCase lifecycle, setup/teardown and assertion mechanism.

Experiment 2 – unittest Assertions and Exception Testing

Aim: To validate different conditions using unittest assertions and verify that expected exceptions are raised.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a test class.
2. Use assertEquals, assertNotEqual, assertTrue, assertFalse and assertIn.
3. Use assertRaises for invalid operations.
4. Run the test module.

Source Code

```
import unittest

def divide(a, b):
    return a / b

class TestAssertions(unittest.TestCase):

    def test_basic_assertions(self):
        self.assertEqual(10 + 5, 15)
        self.assertNotEqual(10, 20)
        self.assertTrue(10 > 2)
        self.assertFalse(2 > 10)
        self.assertIn("Python", "Python Automation")

    def test_exception(self):
        with self.assertRaises(ZeroDivisionError):
            divide(10, 0)

if __name__ == "__main__":
    unittest.main()
```

Expected Result: Expected result: All assertion checks pass and the ZeroDivisionError is correctly recognized.

Conclusion: This experiment demonstrates different validation styles available in unittest.

Experiment 3 – Basic PyTest Test Cases

Aim: To create concise PyTest tests using normal Python functions and the assert statement.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Install PyTest.
2. Create a file named test_math.py.
3. Write functions whose names start with test_.
4. Use Python assert statements for validation.
5. Run pytest with verbose output.

Source Code

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def test_add():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(10, 4) == 6

def test_zero():
    assert add(10, 0) == 10

# Run:
# pytest -v
```

Expected Result: Expected result: PyTest discovers all test functions and displays their pass/fail status.

Conclusion: This experiment shows the concise syntax that makes PyTest convenient for Python automation.

Experiment 4 – PyTest Fixtures and Teardown

Aim: To create a reusable PyTest fixture and use it for setup and teardown of a test resource.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a fixture using `@pytest.fixture`.
2. Prepare a reusable object before the test.
3. Use `yield` to return the object.
4. Place cleanup operations after `yield`.
5. Pass the fixture name into test functions.

Source Code

```
import pytest

@pytest.fixture
def user():
    print("Setup: create test user")

    data = {
        "name": "Saheli",
        "role": "Tester"
    }

    yield data

    print("Teardown: remove test user")

def test_user_name(user):
    assert user["name"] == "Saheli"

def test_user_role(user):
    assert user["role"] == "Tester"

# Run:
# pytest -v -s
```

Expected Result: Expected result: Setup runs before the test and teardown runs after the test.

Conclusion: This experiment demonstrates reusable test lifecycle management using fixtures.

Experiment 5 – Fixture Scope and conftest.py

Aim: To centralize reusable fixtures in `conftest.py` and demonstrate fixture availability without explicit imports.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a `conftest.py` file in the test directory.
2. Define a fixture inside it.
3. Create a separate test file.
4. Use the fixture by adding its name as a function parameter.
5. Run the tests from the directory.

Source Code

```
# conftest.py
import pytest

@pytest.fixture
def browser_name():
    print("Preparing browser configuration")
    return "Chrome"

# test_browser.py
def test_browser_name(browser_name):
    assert browser_name == "Chrome"

def test_browser_available(browser_name):
    assert browser_name in ["Chrome", "Firefox", "Edge"]

# Run:
# pytest -v -s
```

Expected Result: Expected result: Both tests can access browser_name without importing it.

Conclusion: This experiment demonstrates how conftest.py provides shared test configuration and fixtures.

Experiment 6 – PyTest Parameterization

Aim: To execute the same test logic with multiple input values using @pytest.mark.parametrize.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Import pytest.
2. Create a parameterized test function.
3. Provide argument names and multiple input tuples.
4. Validate the expected result for each test case.
5. Run the test file.

Source Code

```
import pytest

@pytest.mark.parametrize(
    "a,b,expected",
    [
        (2, 3, 5),
        (10, 20, 30),
        (-2, 5, 3),
        (0, 7, 7),
        (100, 50, 150)
    ]
)
def test_add(a, b, expected):
    assert a + b == expected

# Run:
# pytest -v
```

Expected Result: Expected result: PyTest creates a separate test invocation for every data set.

Conclusion: This experiment demonstrates data-driven testing without duplicating test code.

Experiment 7 – Selenium Automation with a PyTest Fixture

Aim: To integrate Selenium WebDriver with PyTest by creating the browser in a fixture and ensuring cleanup after each test.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a WebDriver fixture.
2. Launch Chrome before the test.
3. Yield the driver to the test.
4. Open a practice web page.
5. Validate the page title.
6. Quit the browser after the test.

Source Code

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    browser = webdriver.Chrome()
    yield browser
    browser.quit()

def test_example_title(driver):
    driver.get("https://example.com")

    assert "Example Domain" in driver.title

def test_example_heading(driver):
    driver.get("https://example.com")

    heading = driver.find_element(
        "tag name", "h1"
    )

    assert heading.text == "Example Domain"

# Run:
# pytest -v
```

Expected Result: Expected result: A browser opens for each test, the validations pass, and the browser is closed automatically.

Conclusion: This experiment demonstrates clean integration between PyTest fixtures and Selenium.

Experiment 8 – Data-Driven Login Test with Parameterization

Aim: To run multiple login scenarios through one Selenium test using parameterized credentials and expected outcomes.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Define valid and invalid credential combinations.
2. Parameterize username, password and expected result.
3. Open the SauceDemo login page.
4. Enter credentials and submit the form.
5. Compare the resulting URL or error state with the expected result.

Source Code

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By

@pytest.mark.parametrize(
    "username,password,valid",
    [
        ("standard_user", "secret_sauce", True),
        ("wrong_user", "wrong_pass", False),
        ("standard_user", "wrong_pass", False)
    ]
)
def test_login(username, password, valid):
    driver = webdriver.Chrome()

    try:
        driver.get("https://www.saucedemo.com/")

        driver.find_element(
            By.ID, "user-name"
        ).send_keys(username)

        driver.find_element(
            By.ID, "password"
        ).send_keys(password)

        driver.find_element(
            By.ID, "login-button"
        ).click()

        logged_in = "/inventory.html" in driver.current_url

        assert logged_in is valid

    finally:
        driver.quit()
```

Expected Result: Expected result: The valid credential case reaches the inventory page, while invalid cases do not.

Conclusion: This experiment combines PyTest parameterization, Selenium and browser cleanup for data-driven UI testing.

Experiment 9 – Page Object Model with PyTest and Selenium

Aim: To create a maintainable Selenium test using the Page Object Model, separating locators and page actions from the test case.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create a LoginPage class.
2. Store page locators as class attributes.
3. Implement reusable methods for username, password and login.
4. Create a PyTest driver fixture.
5. Use the page object from the test.
6. Verify successful navigation.

Source Code

```
# login_page.py
from selenium.webdriver.common.by import By

class LoginPage:
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")

    def __init__(self, driver):
        self.driver = driver

    def login(self, username, password):
        self.driver.find_element(
            *self.USERNAME
        ).send_keys(username)

        self.driver.find_element(
            *self.PASSWORD
        ).send_keys(password)

        self.driver.find_element(
            *self.LOGIN_BUTTON
        ).click()

# test_login.py
import pytest
from selenium import webdriver
from login_page import LoginPage

@pytest.fixture
def driver():
    browser = webdriver.Chrome()
    yield browser
    browser.quit()

def test_valid_login(driver):
    driver.get("https://www.saucedemo.com/")

    page = LoginPage(driver)
    page.login("standard_user", "secret_sauce")

    assert "/inventory.html" in driver.current_url
```

Expected Result: Expected result: The test logs in successfully while the test itself contains no direct locator implementation.

Conclusion: This experiment demonstrates separation of test behavior from page implementation.

Experiment 10 – PyTest Markers, Skipping and HTML Reporting

Aim: To organize tests using markers, demonstrate conditional test execution and generate an HTML test report.

Requirements: Python 3 and the packages/tools specified in the environment section.

Procedure

1. Create smoke and regression tests.
2. Apply pytest markers to selected tests.
3. Skip a test when a feature is unavailable.
4. Run only the required marker group.
5. Generate an HTML report using pytest-html.

Source Code

```
import pytest

@pytest.mark.smoke
def test_home_page():
    assert True

@pytest.mark.regression
def test_checkout():
    assert True

@pytest.mark.skip(reason="Feature not available")
def test_future_feature():
    assert False

@pytest.mark.xfail(reason="Known defect")
def test_known_issue():
    assert False

# Run smoke:
# pytest -m smoke -v

# Run regression:
# pytest -m regression -v

# Generate HTML report:
# pytest -v --html=report.html --self-contained-html
```

Expected Result: Expected result: Marker selection executes only the requested group, skipped/xfail tests are reported appropriately, and an HTML report is generated.

Conclusion: This experiment demonstrates practical test organization and reporting for a growing automation suite.

Overall Module 2 Lab Conclusion

The practical work in Module 2 covers the complete testing workflow from basic unit tests to maintainable UI automation. The experiments demonstrate unittest, PyTest assertions, fixtures, conftest.py, parameterization, Selenium integration, data-driven testing, Page Object Model, markers and HTML reporting. Together these techniques provide the structure required to create reusable, scalable and maintainable Python automation suites.

Lab Submission Checklist

Experiment	Practical Evidence
1	unittest TestCase + setup/teardown
2	unittest assertions + exception handling
3	Basic PyTest functions + assert
4	PyTest fixture + yield teardown
5	conftest.py reusable fixture
6	Parameterization
7	Selenium + PyTest fixture
8	Data-driven Selenium login
9	Page Object Model
10	Markers + skipping + HTML report